HyperV Agent API Documentation for LLM

📋 Overview

HyperV Agent REST API for local Hyper-V management applications. Supports VM/Container management, networking, and storage operations via multiple backends (HCS, WMI, HCN).

Base URL: `http://localhost:5000/api/v1`
Content-Type: `application/json`
Architecture: Multi-backend (HCS for containers, WMI for VMs, HCN for networking)

🔧 Service Endpoints

Health Check

```
GET /api/v1/service/health
```

Response: 200 OK

```
{
  "status": "healthy",
  "timestamp": "2025-01-11T14:30:00Z",
  "version": "1.0.0",
  "services": {
    "hcs": "available",
    "wmi": "available",
    "hcn": "available"
  }
}
```

Agent Information

```
GET /api/v1/service/info
```

Response: 200 OK

```
{
  "name": "HyperV Agent",
  "version": "1.0.0",
  "description": "HyperV management agent providing REST API for VM, container,
network and storage operations",
```

```
    "endpoints": {
      "vms": "/api/v1/vms",
      "containers": "/api/v1/containers",
      "networks": "/api/v1/networks",
      "storage": "/api/v1/storage"
    },
    "capabilities": [
      "VM management (HCS & WMI)",
      "Container management",
      "Network management",
      "Storage management",
      "Snapshot operations",
      "Replication services"
    ]
  }
```

💻 Virtual Machine Management

List All VMs

```
GET /api/v1/vms
```

Response: 200 OK

```
{
  "HCS": {
    "Count": 0,
    "VMs": [],
    "Backend": "HCS"
  },
  "WMI": {
    "Count": 1,
    "VMs": [
      {
        "Name": "TestVM",
        "State": "Running",
        "Id": "vm-123"
      }
    ],
    "Backend": "WMI"
  }
}
```

Check VM Presence

```
GET /api/v1/vms/{vmName}/present
```

Response: 200 OK

```
{
  "present": true,
  "hcs": false,
  "wmi": true
}
```

Create VM

```
POST /api/v1/vms
```

Request Body:

```
{
  "Id": "vm-unique-id",
  "Name": "MyTestVM",
  "MemoryMB": 2048,
  "CpuCount": 2,
  "DiskSizeGB": 20,
  "Mode": "WMI"
}
```

Response: 200 OK (returns JSON from backend)

VM State Management

```
POST /api/v1/vms/{vmName}/start     # Start VM
POST /api/v1/vms/{vmName}/stop      # Stop VM
POST /api/v1/vms/{vmName}/shutdown  # Graceful shutdown
POST /api/v1/vms/{vmName}/terminate # Force terminate
POST /api/v1/vms/{vmName}/pause     # Pause VM
POST /api/v1/vms/{vmName}/resume    # Resume VM
POST /api/v1/vms/{vmName}/save      # Save state (WMI only)
```

Response: 200 OK, 404 Not Found, 501 Not Implemented (HCS limitations)

Get VM Properties

```
GET /api/v1/vms/{vmName}/properties
```

Response: 200 OK

```
{
  "backend": "WMI",
  "properties": {
    "name": "MyTestVM",
    "state": "Running",
    "memory": 2048,
    "processors": 2
  }
}
```

Configure VM

```
POST /api/v1/vms/{vmName}/configure
```

Request Body:

```
{
  "StartupMemoryMB": 4096,
  "CpuCount": 4,
  "Notes": "Production VM",
  "EnableDynamicMemory": true,
  "MinimumMemoryMB": 512,
  "MaximumMemoryMB": 8192,
  "TargetMemoryBuffer": 20
}
```

VM Snapshots

```
GET /api/v1/vms/{vmName}/snapshots                    # List snapshots
POST /api/v1/vms/{vmName}/snapshots                   # Create snapshot
DELETE /api/v1/vms/{vmName}/snapshots/{snapshotId}    # Delete snapshot
POST /api/v1/vms/{vmName}/snapshots/{snapshotId}/revert # Revert to snapshot
```

VM Storage Management

```
```

```
GET /api/v1/vms/{vmName}/storage/devices              # List storage devices
POST /api/v1/vms/{vmName}/storage/devices             # Add storage device
DELETE /api/v1/vms/{vmName}/storage/devices/{deviceId} # Remove storage device
GET /api/v1/vms/{vmName}/storage/controllers          # List storage
controllers
```

📦 Container Management

Create Container

```
POST /api/v1/containers
```

Request Body:

```
{
  "Id": "container-unique-id",
  "Name": "MyContainer",
  "Image": "mcr.microsoft.com/windows/nanoserver:ltsc2022",
  "MemoryMB": 1024,
  "CpuCount": 1,
  "StorageSizeGB": 10,
  "Mode": "HCS",
  "Environment": {
    "ENV_VAR": "value"
  },
  "PortMappings": {
    "80": "8080"
  }
}
```

List All Containers

```
GET /api/v1/containers
```

Response: 200 OK

```
{
  "HcsContainers": [],
  "WmiContainers": [],
  "Message": "Container listing not yet fully implemented"
}
```

Container Operations

```
GET /api/v1/containers/{containerId}           # Get container info
POST /api/v1/containers/{containerId}/start    # Start container
POST /api/v1/containers/{containerId}/stop     # Stop container
POST /api/v1/containers/{containerId}/terminate # Terminate container
POST /api/v1/containers/{containerId}/pause    # Pause container
POST /api/v1/containers/{containerId}/resume   # Resume container
DELETE /api/v1/containers/{containerId}        # Delete container
```

🌐 Network Management (HCN)

Create NAT Network

```
POST /api/v1/networks/nat?name={networkName}&prefix=192.168.100.0/24
```

Response: 200 OK

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

Network Operations

```
DELETE /api/v1/networks/{networkId}                    # Delete network
GET /api/v1/networks/{networkId}/properties            # Get network properties
```

Endpoint Management

```
POST /api/v1/networks/{networkId}/endpoints            # Create endpoint
DELETE /api/v1/networks/endpoints/{endpointId}         # Delete endpoint
GET /api/v1/networks/endpoints/{endpointId}/properties # Get endpoint
properties
```

Create Endpoint:

```
POST /api/v1/networks/{networkId}/endpoints?
name=endpoint1&ipAddress=192.168.100.10
```

💾 Storage Management

VHD Operations

```
POST /api/storage/vhd                  # Create VHD
PUT /api/storage/vhd/attach            # Attach VHD to VM
PUT /api/storage/vhd/detach            # Detach VHD from VM
PUT /api/storage/vhd/resize            # Resize VHD
```

Create VHD Request:

```
{
  "Path": "C:\\VMs\\test.vhdx",
  "MaxInternalSize": 10737418240,
  "Format": "VHDX",
  "Type": "Dynamic"
}
```

Advanced VHD Operations

```
GET /api/storage/vhd/metadata?vhdPath=C:\VMs\test.vhdx  # Get VHD metadata
PUT /api/storage/vhd/metadata                           # Set VHD metadata
POST /api/storage/vhd/compact                          # Compact VHD
POST /api/storage/vhd/convert                          # Convert VHD
```

Change Tracking

```
POST /api/storage/vhd/{vhdPath}/tracking/enable    # Enable change tracking
POST /api/storage/vhd/{vhdPath}/tracking/disable   # Disable change tracking
GET /api/storage/vhd/{vhdPath}/changes              # Get disk changes
```

📝 Job Management

Storage Job Operations

```
GET /api/v1/jobs/storage                          # List all storage jobs
GET /api/v1/jobs/storage/{jobId}                  # Get job details
GET /api/v1/jobs/storage/{jobId}/affected-elements # Get affected elements
```

```
POST /api/v1/jobs/storage/{jobId}/cancel          # Cancel job
DELETE /api/v1/jobs/storage/{jobId}               # Delete completed job
```

Job Response Example:

```
{
  "JobId": "job-123",
  "OperationType": "CreateVHD",
  "State": "Completed",
  "StartTime": "2025-01-11T14:00:00Z",
  "EndTime": "2025-01-11T14:05:00Z",
  "PercentComplete": 100,
  "Description": "Create VHD job"
}
```

🏗 Common Data Models

VM Creation Request

```
interface CreateVmRequest {
  Id: string;              // Unique identifier
  Name: string;            // VM display name
  MemoryMB: number;        // Memory in MB
  CpuCount: number;        // CPU count
  DiskSizeGB: number;      // Disk size in GB
  Mode: "HCS" | "WMI";     // Backend mode
}
```

Container Creation Request

```
interface CreateContainerRequest {
  Id: string;                           // Unique identifier
  Name: string;                         // Container name
  Image: string;                        // Container image
  MemoryMB: number;                     // Memory limit in MB
  CpuCount: number;                     // CPU count
  StorageSizeGB: number;                // Storage size in GB
  Mode: "HCS" | "WMI";                  // Backend mode
  Environment: Record<string, string>;  // Environment variables
  PortMappings: Record<number, number>; // Port mappings
  VolumeMounts: Record<string, string>; // Volume mounts
}
```

Storage Device Response

```
interface StorageDeviceResponse {
  DeviceId: string;           // Device identifier
  Name: string;               // Device name
  DeviceType: string;         // Type (e.g., "VirtualDisk")
  Path: string;               // File path
  ControllerId: string;       // Controller ID
  ControllerType: string;     // Controller type (e.g., "SCSI")
  IsReadOnly: boolean;        // Read-only status
  OperationalStatus: string;  // Current status
}
```

🚀 LLM Implementation Guide

For Building Local Hyper-V Management App

1. Service Discovery

```
// Check if HyperV Agent is running
fetch('http://localhost:5000/api/v1/service/health')
  .then(response => response.json())
  .then(data => {
    if (data.status === 'healthy') {
      console.log('HyperV Agent is available');
      // Proceed with app initialization
    }
  });
```

2. VM Management Dashboard

```
// Get list of all VMs
fetch('http://localhost:5000/api/v1/vms')
  .then(response => response.json())
  .then(data => {
    const allVMs = [...data.HCS.VMs, ...data.WMI.VMs];
    // Display VMs in UI
  });

// Start VM
function startVM(vmName) {
  fetch(`http://localhost:5000/api/v1/vms/${vmName}/start`, {
    method: 'POST'
  })
  .then(response => {
```

```
    if (response.ok) {
      console.log(`VM ${vmName} started successfully`);
    }
  });
}
```

3. Network Management

```
// Create NAT network for VMs
function createNATNetwork(name, subnet = '192.168.100.0/24') {
  fetch(`http://localhost:5000/api/v1/networks/nat?
name=${name}&prefix=${subnet}`, {
    method: 'POST'
  })
  .then(response => response.json())
  .then(data => {
    console.log('Network created with ID:', data.id);
  });
}
```

4. Container Management

```
// Create Windows container
function createContainer(name, image =
'mcr.microsoft.com/windows/nanoserver:ltsc2022') {
  const containerConfig = {
    Id: `container-${Date.now()}`,
    Name: name,
    Image: image,
    MemoryMB: 1024,
    CpuCount: 1,
    StorageSizeGB: 10,
    Mode: "HCS",
    Environment: {},
    PortMappings: {},
    VolumeMounts: {}
  };

  fetch('http://localhost:5000/api/v1/containers', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(containerConfig)
  });
}
```

5. Storage Operations

```
// Create new VHD
function createVHD(path, sizeGB = 10) {
  const vhdConfig = {
    Path: path,
    MaxInternalSize: sizeGB * 1024 * 1024 * 1024,
    Format: "VHDX",
    Type: "Dynamic"
  };

  fetch('http://localhost:5000/api/storage/vhd', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(vhdConfig)
  });
}
```

🔒 Error Handling

Standard HTTP Status Codes

- **200 OK** - Successful operation
- **202 Accepted** - Async operation started
- **400 Bad Request** - Invalid request data
- **404 Not Found** - Resource not found
- **500 Internal Server Error** - Backend service error
- **501 Not Implemented** - Feature not supported by backend

Error Response Format

```
{
  "error": "Detailed error message",
  "details": "Technical details (in development mode)",
  "backend": "HCS|WMI|HCN"
}
```

🎨 UI Implementation Suggestions

Dashboard Components

1. **Service Status Panel** - Health check results
2. **VM Management Grid** - List/start/stop/configure VMs
3. **Container Panel** - Container lifecycle management
4. **Network Topology** - Visual network management
5. **Storage Overview** - VHD management and monitoring

6. **Job Monitor** - Track long-running operations

Recommended Tech Stack

- **Frontend:** HTML5, CSS3, JavaScript (or React/Vue/Angular)
- **HTTP Client:** Fetch API or Axios
- **UI Framework:** Bootstrap, Tailwind, or Material-UI
- **Charts:** Chart.js for resource monitoring
- **Real-time:** WebSocket or polling for status updates

📊 Monitoring & Status

Real-time VM Monitoring

```
// Poll VM status every 5 seconds
setInterval(async () => {
  const response = await fetch('http://localhost:5000/api/v1/vms');
  const vms = await response.json();
  updateVMDashboard(vms);
}, 5000);
```

Job Status Tracking

```
// Monitor storage jobs
async function monitorStorageJobs() {
  const response = await fetch('http://localhost:5000/api/v1/jobs/storage');
  const jobs = await response.json();

  jobs.filter(job => job.State === 'Running')
      .forEach(job => {
        console.log(`Job ${job.JobId}: ${job.PercentComplete}%`);
      });
}
```

⚠️ Important Notes

Backend Limitations

- **HCS VMs**: Container-like, limited snapshot support, not visible in Hyper-V Manager
- **WMI VMs**: Full-featured traditional VMs, visible in Hyper-V Manager
- **HCN Networks**: Modern networking stack for containers and VMs
- **Storage Jobs**: Async operations for VHD management

Security Considerations

- API runs on localhost only
- Requires elevated privileges (Administrator)
- No authentication (local management tool)
- Direct hardware access through Hyper-V APIs

Development Tips

- Use health check endpoint to verify connectivity
- Implement polling for long-running operations
- Handle 501 errors gracefully (unsupported features)
- Test with both HCS and WMI backends
- Monitor job endpoints for storage operations

📋 Quick Start Checklist

1. ☑ Check service health (`/service/health`)
2. ☑ Get agent capabilities (`/service/info`)
3. ☑ List existing VMs (`/vms`)
4. ☑ Create test VM (`POST /vms`)
5. ☑ Test VM operations (start/stop)
6. ☑ Create network for VMs (`/networks/nat`)
7. ☑ Test container operations
8. ☑ Monitor jobs (`/jobs/storage`)

This API documentation provides everything needed for an LLM to create a comprehensive local Hyper-V management application with modern web technologies.